

Variabili, funzioni e visibilità

Variabili

- Una variabile corrisponde a un insieme di celle di memoria che contengono un valore modificabile dal programma.
- Il numero di celle occupate dipende dal tipo della variabile.
- Prima di poter essere usata, una variabile DEVE essere **dichiarata**:
tipo nome_della_variabile;
- È possibile dichiarare più variabili dello stesso tipo sulla stessa riga:
tipo variabile_1, variabile_2;

Dove dichiarare le variabili?

- Le variabili possono essere dichiarate:
 - dentro una funzione o un blocco di istruzioni (variabile locale);
 - nella lista dei parametri della funzione;
 - all'esterno di tutte le funzioni (variabile globale).

Esempio:

```
int variabile_globale = 5;

int f1(int parametro) {
    return parametro + 1;
}

int main(void) {
    int variabile_locale = f1(variabile_globale) + 1;
    return 0;
}
```

Variabili locali (o private)

- Le variabili locali sono dichiarate e definite all'interno di una funzione.
- La loro vita inizia nel momento della dichiarazione/definizione e termina all'uscita della funzione, quando la funzione chiamata restituisce il controllo alla funzione chiamante.
- Poiché compaiono e scompaiono in base alle chiamate alle funzioni, sono dette **automatiche**.
- Devono essere **inizializzate esplicitamente**.
- Sono *accessibili* solo all'interno della funzione (o del blocco) in cui sono dichiarate.
- I **parametri** di una funzione e le variabili dentro blocchi di codice della funzione stessa sono anch'essi variabili locali.

Variabili locali - Esempio

```
float contante; // Contante nel borsellino (INIZIALMENTE 0, LA VARIABILE è GLOBALE)
void preleva();
```

.....

poiché **p** è definita nella funzione, non è accessibile da nessun'altra parte

```
void preleva(){
    float p = leggi_float("Quanti soldi vorresti prelevare? ");
    if (p<0)
        printf("Operazione invalida. Non puoi prelevare un ammontare negativo\n\n");
    else
        if (p<=contante){
            printf("Hai prelevato %.2f euro dei %.2f disponibili.\n\n", p, contante);
            contante -= p;
        }
        else
            printf("Non hai denaro sufficiente nel borsellino.\n\n");
}
```

Cosa succede se si compila questo codice?

```
#include <stdio.h>

void f1();
void f2();

int main(){
    f1();
    f2();
    printf("dentro il main il valore di p=%f", p);
}

void f1(){
    float p = 100;
    printf("dentro f1 il valore di p=%f", p);
}

void f2(){
    printf("dentro f2 il valore di p=%f", p);
}
```

Cosa succede se si compila questo codice?

```
#include <stdio.h>

void f1();
void f2();

int main(){
    f1();
    f2();
    printf("dentro il main il valore di p=%f", p);
}

void f1(){
    float p = 100;
    printf("dentro f1 il valore di p=%f", p);
}

void f2(){
    printf("dentro f2 il valore di p=%f", p);
}
```

Esempio02.c:11:50: error: use of undeclared identifier 'p'

```
printf("dentro il main il valore di p e' %f",p);
```

^

Esempio02.c:21:46: error: use of undeclared identifier 'p'

```
printf("dentro f2 il valore di p e' %f", p);
```

^

2 errors generated.

Variabili esterne (o globali)

- Le variabili globali sono definite una sola volta al di fuori di qualsiasi funzione.
- La loro vita inizia nel momento della definizione e termina all'uscita del programma.
- Sono **inizializzate** a valori di default.
- Sono **accessibili** da qualsiasi funzione.
- Sono utili quando più funzioni devono condividere il valore in essa memorizzato ma non se ne deve abusare perché rendono il codice meno chiaro ed estensibile. Inoltre, essendo permanenti, occupano memoria per tutta la durata dell'esecuzione del programma.

Variabili esterne - Esempio

```
float contante; // Contante nel borsellino (INIZIALMENTE 0, LA VARIABILE è GLOBALE)
void preleva();
```

.....

poiché **p** è definita fuori da tutte le funzioni, è accessibile ovunque

```
void preleva(){
    float p = leggi_float("Quanti soldi vorresti prelevare? ");
    if (p<0)
        printf("Operazione invalida. Non puoi prelevare un ammontare negativo\n\n");
    else
        if (p<=contante){
            printf("Hai prelevato %.2f euro dei %.2f disponibili.\n\n", p, contante);
            contante -= p;
        }
        else
            printf("Non hai denaro sufficiente nel borsellino.\n\n");
}
```

Variabili esterne - Esempio

```
#include <stdio.h>
```

```
void f1();
```

```
void f2();
```

```
float p = 0;
```

```
int main(){
```

```
    f1();
```

```
    f2();
```

```
    printf("dentro il main il valore di p=%f\n", p);
```

```
}
```

```
void f1(){
```

```
    p = 100;
```

```
    printf("dentro f1 il valore di p=%f\n", p);
```

```
}
```

```
void f2(){
```

```
    p -= 20;
```

```
    printf("dentro f2 il valore di p=%f\n", p);
```

```
}
```

Quando lo eseguiamo, otteniamo:

dentro f1 il valore di p=100.000000

dentro f2 il valore di p=80.000000

dentro il main il valore di p=80.000000

Variabili esterne: dichiarazione

- Se una funzione vuole accedere ad una variabile esterna, deve dichiararla in modo da rendere esplicito il tipo e migliorare il controllo dei tipi da parte del compilatore
- La dichiarazione può essere *implicita* se la variabile è definita nello stesso file sorgente in cui si trova la funzione che la usa.
- La dichiarazione deve essere *esplicita* se la variabile è definita in un altro file. In questo caso deve essere utilizzata la parola chiave **extern**.

```
int max = 100; // qui la variabile max è definita (creata in memoria)

int main(void) {
    extern int max; //qui la variabile max è dichiarata
    // ...
}
```

Definizione vs dichiarazione

- **Definizione** di una variabile:
 - è il processo con il quale si crea la variabile in memoria.
- **Dichiarazione** di una variabile:
 - permette di asserire il tipo della variabile senza allocare in memoria.
- In pratica:
 - per le variabili locali, definizione e dichiarazione vengono fatte simultaneamente.
 - per le variabili esterne, la definizione viene fatta nel punto in cui per la prima volta la variabile viene definita; la dichiarazione (implicita o esplicita) viene fatta ogni volta che si dichiara l'uso (e quindi il tipo) della variabile nella funzione.

Dichiarazione implicita

```
#include <stdio.h>

void f1();
void f2();

float p = 0;
int main(){
    f1();
    f2();
    printf("dentro il main il valore di p=%f\n", p);
}

void f1(){
    p = 100;
    printf("dentro f1 il valore di p=%f\n", p);
}

void f2(){
    p -=20;
    printf("dentro f2 il valore di p=%f\n", p);
}
```

Si può usare la dichiarazione implicita se la definizione è fatta nello stesso file.

Dichiarazione esplicita

```
#include <stdio.h>

void f1();
void f2();

float p = 0;

int main(){
    extern float p;
    f1();
    f2();
    printf("dentro il main il valore di p=%f\n", p);
}

void f1(){
    extern float p;
    p = 100;
    printf("dentro f1 il valore di p=%f\n", p);
}

void f2(){
    extern float p;
    p -=20;
    printf("dentro f2 il valore di p=%f\n", p);
}
```

Variabili locali, esempio

```
void disegna quadrato(int lato){  
    for (int i = 1; i<=lato; i++){  
        for (int j = 1; j<=lato; j++){  
            printf("*");  
            printf("\n");  
        }  
    }  
}
```

- **lato** è un parametro, come tale è una variabile locale visibile in tutta la funzione.
- **i** è una variabile locale visibile dentro il blocco del ciclo esterno.
- **j** è una variabile locale visibile dentro il blocco del ciclo interno.

Oscuramento delle variabili (Shadowing)

- Anche se non è buona norma, è **possibile chiamare più variabili in blocchi diversi con lo stesso nome**.
- In tal caso si parla di ri-definizione della variabile.
- La nuova variabile non sovrascrive la precedente ma la “oscura” (shadow in inglese).
- All'interno di un blocco, si può accedere solo ad una variabile con un certo nome. Se ci sono più variabili con lo stesso nome, si può accedere solo a quella dichiarata più recentemente.

Funzioni

Modularizzazione dei programmi in C

- In C si utilizzano le funzioni per la modularizzazione dei programmi combinando la scrittura di nuove funzioni con l'utilizzo di funzioni preconfezionate.
- La libreria standard del C fornisce una collezione di funzioni per eseguire comuni calcoli matematici, manipolazioni di stringhe, manipolazioni di caratteri, input/output e molte altre operazioni utili.
- È possibile definire funzioni per eseguire compiti specifici che possono essere **utilizzate in più punti** di un programma.

Funzioni

- In molti casi il main di un programma è costituito da una serie di chiamate a funzione che svolgono la maggior parte del lavoro.
- Motivi:
 - approccio **divide et impera**: rende lo sviluppo di programmi complesso più gestibile;
 - **riusabilità del software**: sviluppare nuovi programmi utilizzando funzioni esistenti.
- Ogni funzione deve limitarsi a svolgere un singolo compito ben definito e il nome della funzione deve esprimere tale compito.
- Se una funzione è troppo complessa, è bene dividerla in funzioni più piccole (**decomposizione**).

Esempio

```
#include <stdio.h>

int square(int number);           // prototipo della funzione

int main(void) {
    for (int x = 1; x <= 10; x++) {
        printf("%d ", square(x)); // chiamata a funzione
    }
    printf("\n");

    return 0;
}

int square(int number) { // number è una copia dell'argomento passato alla funzione
    return number * number;
}
```

Output:

1 4 9 16 25 36 49 64 81 100

Funzioni

- La funzione `square` è invocata all'interno del `main`:

```
printf("%d ", square(x)); // chiamata a funzione
```

- La funzione riceve una copia del valore di `x` come argomento (`number`).
- Il risultato viene calcolato e restituito al `main` utilizzando l'istruzione **`return`**.
- Il risultato restituito dalla funzione **`square`** è poi utilizzato all'interno della chiamata alla funzione **`printf`** che lo visualizza a schermo.

Prototipo di una funzione

- La seguente riga costituisce il **prototipo** della funzione:

```
int square(int number);    // prototipo della funzione
```

- L'**int** tra parentesi informa il compilatore che la funzione si aspetta di ricevere un parametro di tipo **int**.
- L'**int** più a sinistra informa il compilatore che la funzione restituisce un valore di tipo **int**.
- Non inserire il punto e virgola alla fine del **prototipo** di una funzione è un errore di sintassi.
- Quando viene chiamata una funzione, il compilatore controlla se:
 - il numero di argomenti è corretto;
 - gli argomenti sono del tipo corretto.

Definizione di una funzione

- La **definizione** della funzione è la seguente:

```
int square(int number) {  
    return number * number;  
}
```

- Il prototipo della funzione, la prima riga della definizione della funzione e le chiamate alla funzione devono concordare nel numero, nel tipo e nell'ordine degli argomenti e dei parametri.
- Per indicare che la funzione non restituisce un valore si utilizza **void**.
- Inserire il punto e virgola dopo la chiusura della parentesi tonda nella **definizione** di una funzione è un errore.

Header File

- I file header raccolgono una serie di funzioni e variabili che possono essere riutilizzabili in vari punti del programma. Permettono al compilatore di avere tutte le informazioni utili per effettuare i vari controlli.
- Questi file permettono di strutturare meglio il codice e hanno estensione .h
- Un esempio è `stdio.h`, che viene incluso per utilizzare le funzioni di I/O.
- Gli header della libreria standard si includono utilizzando le parentesi angolari.
Esempio:
 - `#include <stdio.h>`
- Gli header creati dal programmatore si includono utilizzando le virgolette.
Esempio:
 - `#include "funzioni.h"`

Strutturare programmi su più file

- I file header contengono le dichiarazioni e vengono inclusi in tutti i file sorgente in cui servono.
- **#include** copia il contenuto dell'header nel file corrente.
- È possibile suddividere il codice in più file sorgente.
- Per esempio, spesso le funzioni ausiliarie o di utility sono inserite in un file a parte con estensione .c
- Al momento della compilazione, si compilano tutti i file sorgente (.c) ma non i file header (.h).

```
gcc main.c funzioni.c -o esempio
```


Esempio

```
// file funzioni.h  
  
int square(int number);
```

```
// file funzioni.c  
  
#include "funzioni.h"  
  
int square(int number) {  
    return number * number;  
}
```

```
// file main.c  
  
#include <stdio.h>  
#include "funzioni.h"  
  
int main(void) {  
    for (int x = 1; x <= 10; x++) {  
        printf("%d ", square(x));  
    }  
    printf("\n");  
  
    return 0;  
}
```

Per evitare che un header sia incluso più volte

```
// file funzioni.h
#ifndef FUNZIONI_H
#define FUNZIONI_H

int square(int number);

#endif
```

```
// file funzioni.c

#include "funzioni.h"

int square(int number) {
    return number * number;
}
```

```
// file main.c

#include <stdio.h>
#include "funzioni.h"

int main(void) {
    for (int x = 1; x <= 10; x++) {
        printf("%d ", square(x));
    }
    printf("\n");

    return 0;
}
```

Per evitare che un header sia incluso più volte

```
// file funzioni.h
#ifndef FUNZIONI_H
#define FUNZIONI_H

int square(int number);

#endif
```

```
// file funzioni.c

#include "funzioni.h"

int square(int number) {
    return number * number;
}
```

La direttiva **#ifndef FUNZIONI_H** fa sì che il codice successivo (fino a **#endif**) sia considerato solo se non è mai stato definito un simbolo chiamato **FUNZIONI_H**.

La prima volta che funzioni.h viene incluso questo è vero, e il simbolo **FUNZIONI_H** viene definito subito dopo.

Se si prova a includere nuovamente funzioni.h, il simbolo **FUNZIONI_H** sarà stato già definito in precedenza, per cui il codice fino a **#endif** non viene incluso nuovamente.

Funzioni matematiche (math.h)

- Per utilizzare le funzioni matematiche si deve includere l'header **math.h**

```
#include <math.h>
```

- In base alla versione del compilatore utilizzata, *potrebbe* essere necessario aggiungere anche il flag `-lm` quando si compilano programmi che utilizzano queste funzioni, oltre ad includere l'header:

```
gcc programma.c -o programma -lm
```

Funzioni matematiche (math.h)

- Ad esempio, per calcolare la radice quadrata di un numero, si può utilizzare la funzione **sqrt**:

```
printf("%g", sqrt(900));
```

- Molte delle funzioni della libreria **math** lavorano con variabili di tipo *double*.
- Ad esempio, la funzione **sqrt** si aspetta un argomento di tipo *double* e restituisce un risultato di tipo *double*.
- Gli argomenti passati alle funzioni possono essere costanti, variabili o espressioni.

Funzioni matematiche (math.h)

Function	Description	Example
sqrt (x)	square root of x	sqrt (900.0) is 30.0 sqrt(9.0) is 3.0
cbrt (x)	cube root of x (C99 and C11 only)	cbrt (27.0) is 3.0
exp (x)	exponential function e^x	exp (1.0) is 2.718282 exp (2.0) is 7.389056
log (x)	natural logarithm of x (base e)	log (2.718282) is 1.0 log (7.389056) is 2.0
log10 (x)	logarithm of x (base 10)	log10 (1.0) is 0.0 log10 (10.0) is 1.0 log10 (100.0) is 2.0
fabs (x)	absolute value of x as a floating-point number	fabs (13.5) is 13.5 fabs (0.0) is 0.0 fabs (-13.5) is 13.5

Funzioni matematiche (math.h)

Function	Description	Example
ceil(x)	rounds x to the smallest integer not less than x	ceil(9.2) is 10.0 ceil(-9.8) is -9.0
floor(x)	rounds x to the largest integer not greater than x	floor(9.2) is 9.0 floor(-9.8) is -10.0
pow(x, y)	x raised to power y (x^y)	pow(2, 7) is 128.0 pow(9, .5) is 3.0
fmod(x, y)	remainder of x/y as a floating-point number	fmod(13.657, 2.333) is 1.992
sin(x)	trigonometric sine of x (x in radians)	sin(0.0) is 0.0
cos(x)	trigonometric cosine of x (x in radians)	cos(0.0) is 1.0
tan(x)	trigonometric tangent of x (x in radians)	tan(0.0) is 0.0

Alcuni header file utili

Header	Explanation
<stdio.h>	Contiene i prototipi di funzione per le funzioni di input/output.
<stdlib.h>	Contiene prototipi di funzioni per la conversione di numeri in testo e di testo in numeri, l'allocazione della memoria, i numeri casuali e altre funzioni di utilità.
<math.h>	Contiene i prototipi delle funzioni della libreria matematica.
<string.h>	Contiene i prototipi delle funzioni di elaborazione delle stringhe.
<time.h>	Contiene prototipi di funzioni e tipi per manipolare l'ora e la data.
<stddef.h>	Contiene le definizioni dei tipi comuni utilizzati da C.
<stdbool.h>	Contiene la definizione del tipo di dato bool e dei valori true e false.

stdbool

- A partire da C99, in C esiste il tipo di dato **_Bool**, che può assumere solo valore 0 oppure 1.
- Includendo l'header **<stdbool.h>**, vengono definiti il tipo **bool** e i valori **true** (1) e **false** (0).

```
#include <stdbool.h>
```

```
int main(void) {  
    bool x = true;  
    bool y = false;  
    // ...  
  
    return 0;  
}
```

Generazione di numeri pseudo-casuali

- La "casualità" può essere introdotta nei programmi C usando la funzione **rand** della libreria standard C, includendo l'header **<stdlib.h>**.

Generazione di numeri pseudo-casuali

- La "casualità" può essere introdotta nei programmi C usando la funzione **rand** della libreria standard C, includendo l'header **<stdlib.h>**.

```
int value = rand();
```

- La funzione rand genera un intero compreso tra **0** e **RAND_MAX** (una costante definita in `stdlib.h`).
- Spesso il range di default non è adatto per gli scopi dell'applicazione (ad esempio potremmo voler generare valori 0 e 1 per simulare il lancio di una moneta, o valori compresi tra 1 e 6 per simulare il lancio di un dado).
- Inoltre, se si genera una sequenza di numeri "casuali" e si esegue il programma più volte, si noterà che la sequenza di numeri è sempre uguale.

Generazione di numeri pseudo-casuali

- Per generare un numero che sia minore di **max**, si può usare l'operatore modulo. La seguente istruzione genera numeri compresi tra 0 e 9:

```
int value = rand() % 10;
```

- Per generare un numero che sia compreso tra **min** e **max** (inclusi) si può usare l'operatore modulo come sopra, sommando il valore di **min**:

```
int casuale(int min, int max) {  
    return min + rand() % (max - min + 1);  
}
```

Generazione di numeri pseudo-casuali

- Per generare un numero che sia minore di **max**, si può usare l'operatore modulo. La seguente istruzione genera numeri compresi tra 0 e 9:

```
int value = rand() % 10;
```

- Per generare un numero che sia compreso tra **min** e **max** (inclusi) si può usare l'operatore modulo come sopra, sommando il valore di **min**:

```
int casuale(int min, int max) {  
    return min + rand() % (max - min + 1);  
}
```

- Ad esempio, per un dado a 6 facce:

```
int lancia_dado(void) {  
    return 1 + rand() % 6;  
}
```

```

#include <stdio.h>
#include <stdlib.h>

#define SIZE 6 + 1

int lancia_dado(void);

int main(void) {
    int contatori[SIZE];
    int valore;

    // Inizializziamo i contatori
    for (size_t i = 0; i < SIZE; i++) {
        contatori[i] = 0;
    }

    // Lanciamo un dado 10 milioni di volte
    for (int i = 0; i < 10000000; i++) {
        valore = lancia_dado();
        contatori[valore]++;
    }

    // Stampiamo i valori dei contatori
    for (size_t i = 1; i < SIZE; i++) {
        printf("%lu: %d\n", i, contatori[i]);
    }

    return 0;
}

int lancia_dado(void) {
    return 1 + rand() % 6;
}

```

Esempio: lancio di un
dado con valori
sempre *uguali* ad
ogni esecuzione

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define SIZE 6 + 1

int lancia_dado(void);

int main(void) {
    int contatori[SIZE];
    int valore;

    srand(time(NULL));

    // Inizializziamo i contatori
    for (size_t i = 0; i < SIZE; i++) {
        contatori[i] = 0;
    }

    // Lanciamo un dado 10 milioni di volte
    for (int i = 0; i < 10000000; i++) {
        valore = lancia_dado();
        contatori[valore]++;
    }

    // Stampiamo i valori dei contatori
    for (size_t i = 1; i < SIZE; i++) {
        printf("%lu: %d\n", i, contatori[i]);
    }

    return 0;
}

int lancia_dado(void) {
    return 1 + rand() % 6;
}

```

Esempio: lancio di un
 dado con valori che
cambiano ad ogni
 esecuzione

Seed e ripetibilità

- Il fatto che il programma generi sempre la stessa sequenza di numeri casuali è una caratteristica voluta, perché consente di ottenere risultati **ripetibili**.
- I numeri generati da **rand()** sono detti pseudo-casuali. Sembrano casuali e garantiscono una serie di proprietà, ma in realtà sono generati tramite un algoritmo, in modo deterministico.
- Per cambiare la sequenza generata si può usare un seed diverso, grazie alla funzione **srand**.
- Passando valori diversi a **srand** si otterrà una sequenza di valori diversi con **rand**.
- Generalmente **srand** deve essere chiamata solo una volta, all'inizio del programma. Una sola chiamata basta per cambiare l'intera sequenza che sarà generata.

Seed e ripetibilità - time

- Per ottenere un seed diverso ad ogni esecuzione del programma, generalmente si utilizza la funzione **time** (includendo l'header **time.h**):

```
srand(time(NULL));
```

- `time(NULL)` restituisce il numero di secondi trascorsi dall'1 Gennaio 1970.

Pila delle chiamate delle funzioni

Come fa il
calcolatore a
eseguire il
codice in
figura senza
confondersi?

```
#include <stdio.h>

void f_simple(int);
void f_complex(int);
void f_nested(int);

int main(){
    printf("\ndentro il main, prima di chiamare f_simple \n");
    f_simple(1);
    printf("\ndentro il main, ho eseguito f_simple, sto chiamando f_complex\n");
    f_complex(1);
    printf("\ndentro il main, ho eseguito f_complex, termina\n\n");
}

void f_simple(int a){
    printf("\tfunzione f_simple, a=%d, ... ora ritorno\n", a);
}

void f_complex(int a){
    printf("\tfunzione f_complex, a=%d \n", a);
    printf("\tfunzione f_complex, sto per chiamare f_nested\n");
    f_nested(a+1);
    printf("\tfunzione f_complex, ho eseguito f_nested, ora ritorno\n");
}

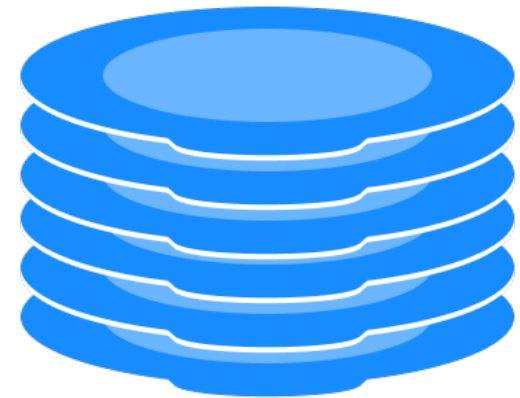
void f_nested(int a){
    printf("\t\tfunzione f_nested, a=%d \n", a);
    printf("\t\tfunzione f_nested, sto per chiamare f_simple\n\t\t");
    f_simple(a+1);
    printf("\t\tfunzione f_nested, ho eseguito f_simple, ora ritorno\n");
}
```

Pila

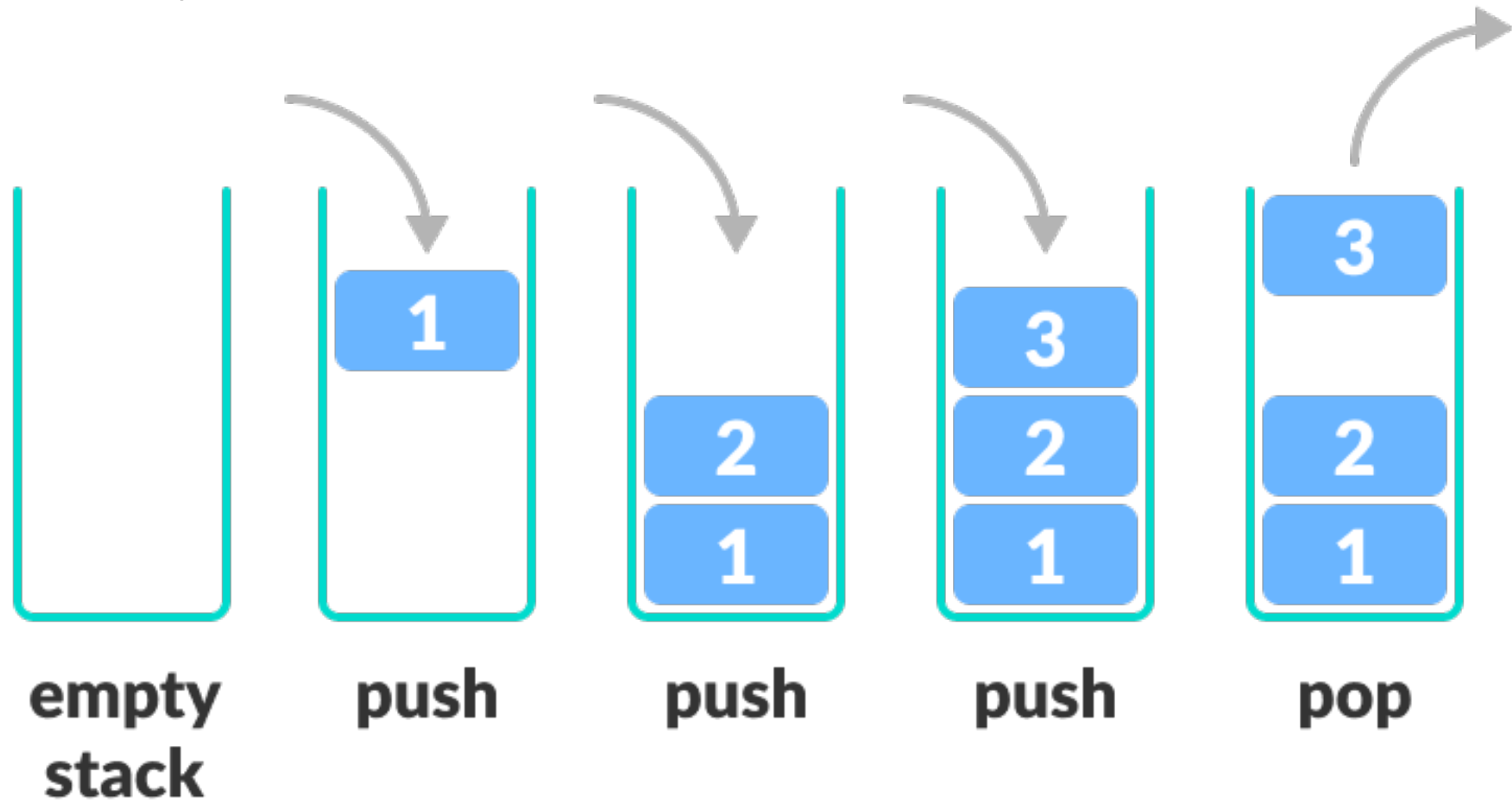
- Una pila è una struttura dati logica con molti utilizzi diversi.
- Possiamo usare il concetto di pila per capire come funziona il meccanismo di shadowing e la chiamata a funzione.
- Pila in inglese si dice stack.
- È importante capire come funziona il cosiddetto **stack di esecuzione**.

Pila

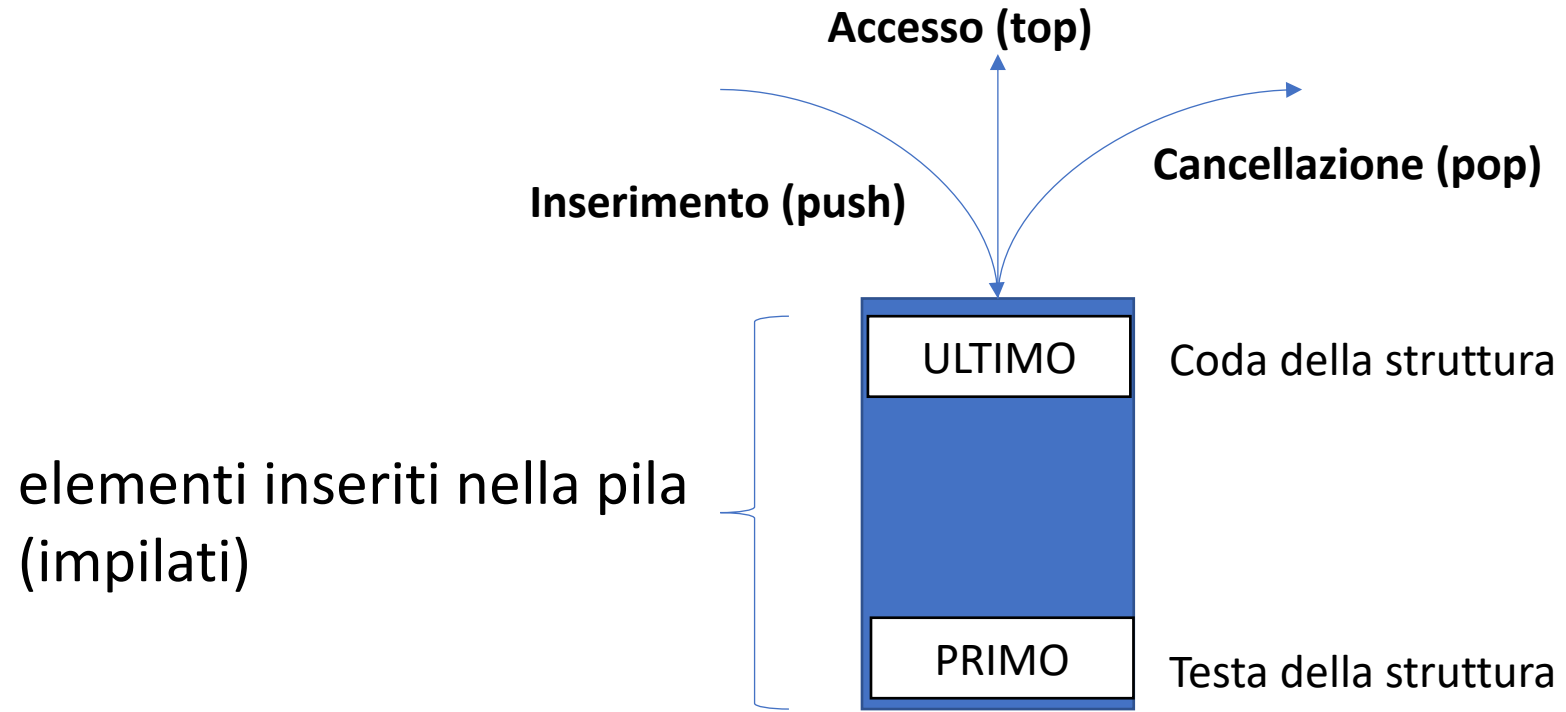
- Le operazioni che si possono fare su una pila sono quelle di:
 - inserimento di un elemento (**push**);
 - estrazione di un elemento (**pop**);
 - accesso ad un elemento.
- La pila è caratterizzata da una politica di accesso ai dati detta **LIFO** (***last-in first-out***), ovvero l'ultimo elemento inserito nella struttura dati è anche il primo elemento a cui si può accedere (e per esempio estrarre).
- Si pensi, ad esempio, ad una pila di piatti.



Pila - Operazioni



Pila



PILA: strategia LIFO

Stack di esecuzione – call stack

- Gli elementi inseriti nello stack di esecuzione sono detti **record di attivazione**.
- **Il call stack** (pila delle chiamate):
 - è una **zona di memoria** di un programma;
 - è organizzata in forma di **pila**;
 - memorizza informazioni sulle subroutine attive in un dato momento (ovvero funzioni invocate la cui esecuzione non è terminata).
- Lo stack serve per **tenere traccia dei punti in cui** ogni subroutine attiva dovrebbe **restituire il controllo** quando termina l'esecuzione.
 - C'è solitamente solo un call stack associato con un programma in esecuzione.

Stack di esecuzione – call stack

- Quando vi è la chiamata ad una funzione (call), si ha che:
 - la funzione chiamante inserisce un elemento in cima allo stack che contiene **l'indirizzo di ritorno** (a cui restituire il controllo), ovvero l'indirizzo dell'istruzione da eseguire quando la funzione chiamata avrà terminato l'esecuzione;
 - viene eseguita la funzione chiamata;
 - al termine della sua esecuzione, la funzione chiamata toglie dalla cima dello stack l'indirizzo di ritorno e trasferisce il controllo a quell'indirizzo (ovvero la prima istruzione da eseguire della funzione chiamante).
- Se viene usato tutto lo spazio a disposizione dello stack si ha un errore detto **stack overflow**.

Stack di esecuzione – call stack

- In alcuni linguaggi, il programmatore può avere accesso alla lista delle chiamate nello stack ma mai alla memoria dello stack stesso (per evitare la corruzione dei dati nello stack).
- In base al linguaggio e al sistema operativo, **lo stack può memorizzare anche** informazioni aggiuntive nel suo record:
 - **variabili locali automatiche;**
 - **parametri;**
 - particolari puntatori (soprattutto in linguaggi ad oggetti...).
- Anche i blocchi di codice in cui sono presenti dichiarazioni di variabili possono generare record di attivazione inseriti in cima allo stack. Questo è il modo in cui si gestisce lo shadowing di variabili.

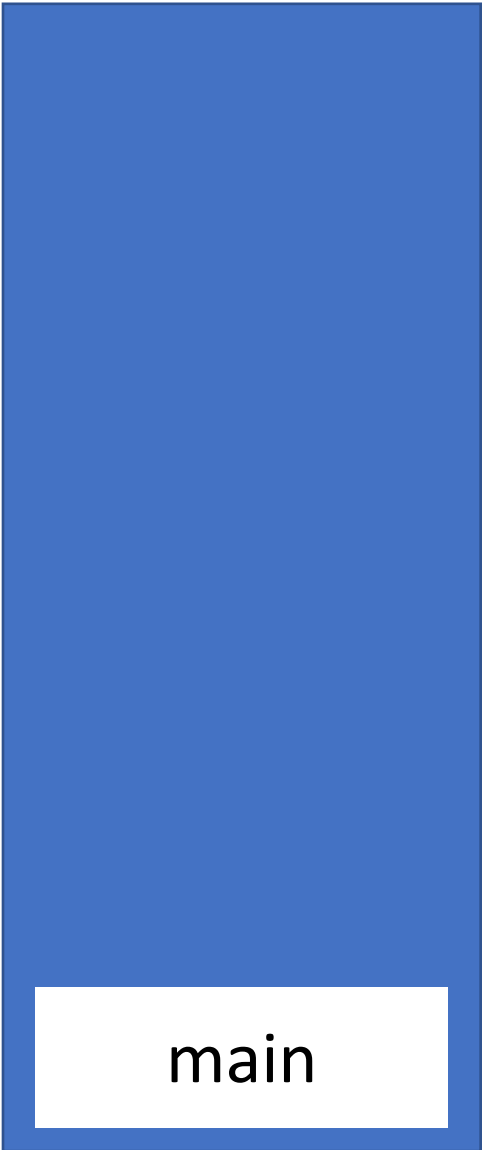
Esempio

Quando eseguiamo il programma, un primo record è aggiunto allo stack

```
→ int main(){  
    printf("\ndentro il main, prima di chiamare f_simple \n");  
    f_simple(1);  
    printf("\ndentro il main, ho eseguito f_simple, sto chiamando f_complex\  
    f_complex(1);  
    printf("\ndentro il main, ho eseguito f_complex, termina\n\n");  
}
```

Eventuali variabili locali sono inserite nel record di attivazione.
In questo caso, nessuna.

STACK



main

Esempio

Eseguiamo le istruzioni una ad una (si veda la freccia a sinistra)

```
int main(){  
→ printf("\ndentro il main, prima di chiamare f_simple \n");  
  f_simple(1);  
  printf("\ndentro il main, ho eseguito f_simple, sto chiamando f_complex\  
  f_complex(1);  
  printf("\ndentro il main, ho eseguito f_complex, termina\n\n");  
}
```

A terminale vediamo

```
dentro il main, prima di chiamare f_simple  
funzione f_simple: a=1 ora ritorno
```

STACK



Esempio

Eseguiamo le istruzioni una ad una (si veda la freccia a sinistra)

```
int main(){  
    printf("\ndentro il main, prima di chiamare f_simple \n");  
→ f_simple(1);  
    printf("\ndentro il main, ho eseguito f_simple, sto chiamando f_complex\  
    f_complex(1);  
    printf("\ndentro il main, ho eseguito f_complex, termina\n\n");  
}
```

Quando invochiamo `f_simple`, un nuovo record viene inserito nello stack, conterrà l'indirizzo dell'istruzione successiva
Inoltre, conterrà il valore del parametro passato

STACK



Esempio

Eseguiamo le istruzioni una ad una (si veda la freccia a sinistra)

```
void f_simple(int a){  
→ printf("\tfunzione f_simple, a=%d, ... ora ritorno\n", a);  
}
```

A terminale vediamo

```
dentro il main, prima di chiamare f_simple  
funzione f_simple, a=1, ... ora ritorno
```

STACK



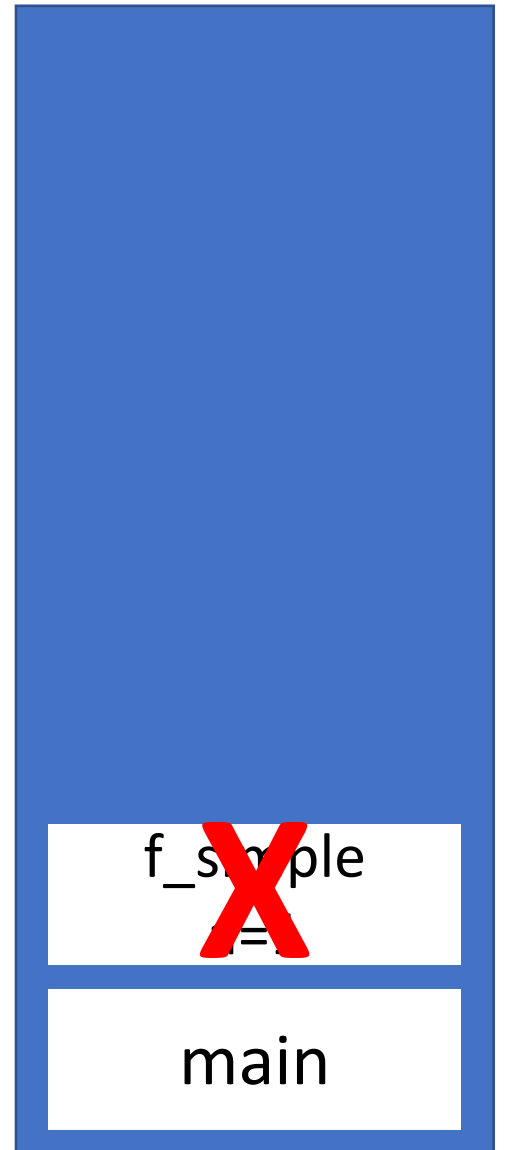
Esempio

Eseguiamo le istruzioni una ad una (si veda la freccia a sinistra)

```
void f_simple(int a){  
    printf("\tfunzione f_simple, a=%d, ... ora ritorno\n", a);  
→ }
```

Quando f_simple termina, il record corrispondente viene eliminato ed il controllo torna al main

STACK



Esempio

Eseguiamo le istruzioni una ad una (si veda la freccia a sinistra)

```
int main(){  
    printf("\ndentro il main, prima di chiamare f_simple \n");  
    f_simple(1);  
→   printf("\ndentro il main, ho eseguito f_simple, sto chiamando f_complex\  
    f_complex(1);  
    printf("\ndentro il main, ho eseguito f_complex, termina\n\n");  
}
```

A terminale vediamo

```
dentro il main, prima di chiamare f_simple  
    funzione f_simple, a=1, ... ora ritorno
```

```
dentro il main, ho eseguito f_simple, sto chiamando f_complex
```

Appena invochiamo f_complex, aggiungiamo un record di attivazione....

STACK

main

Esempio

Eseguiamo le istruzioni una ad una (si veda la freccia a sinistra)

```
void f_complex(int a){  
    printf("\tfunzione f_complex, a=%d \n", a);  
    printf("\tfunzione f_complex, sto per chiamare f_nested\n");  
→ f_nested(a+1);  
    printf("\tfunzione f_complex, ho eseguito f_nested, ora ritorno\n");  
}
```

A terminale avremo

dentro il main, prima di chiamare f_simple
funzione f_simple, a=1, ... ora ritorno

dentro il main, ho eseguito f_simple, sto chiamando f_complex
funzione f_complex, a=1
funzione f_complex, sto per chiamare f_nested

Ed un nuovo record è aggiunto allo stack.

STACK



Esempio

Eseguiamo le istruzioni una ad una (si veda la freccia a sinistra)

```
void f_nested(int a){  
    printf("\t\tfunzione f_nested, a=%d \n", a);  
    printf("\t\tfunzione f_nested, sto per chiamare f_simple\n\t\t");  
→ f_simple(a+1);  
    printf("\t\tfunzione f_nested, ho eseguito f_simple, ora ritorno\n");  
}
```

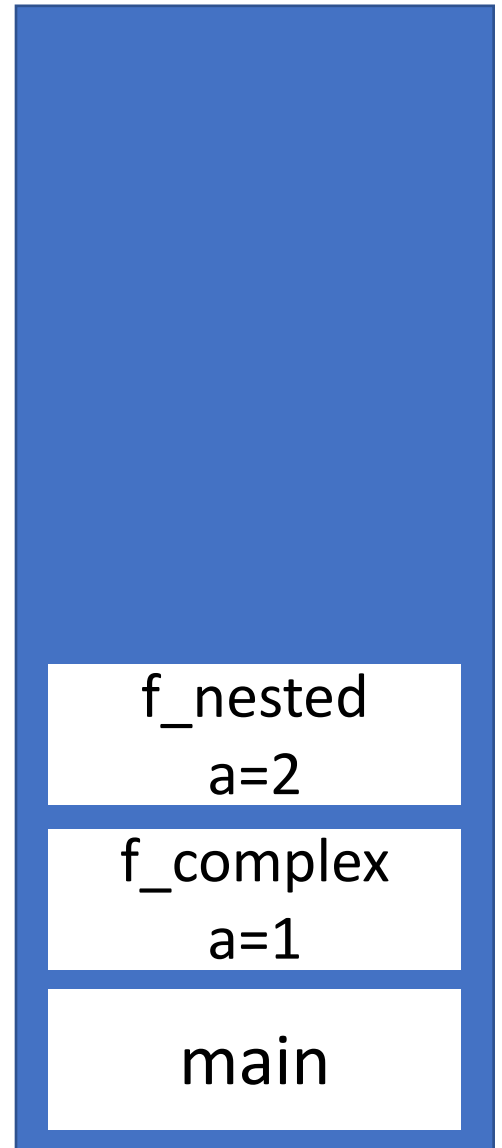
A terminale avremo

```
dentro il main, prima di chiamare f_simple  
funzione f_simple, a=1, ... ora ritorno
```

```
dentro il main, ho eseguito f_simple, sto chiamando f_complex  
funzione f_complex, a=1  
funzione f_complex, sto per chiamare f_nested  
funzione f_nested, a=2  
funzione f_nested, sto per chiamare f_simple  
funzione f_simple, a=3, ... ora ritorno
```

Ed un nuovo record è aggiunto allo stack per f_simple.

STACK



Esempio

Eseguiamo le istruzioni una ad una (si veda la freccia a sinistra)

```
→ void f_simple(int a){  
    printf("\tfunzione f_simple, a=%d, ... ora ritorno\n", a);  
}
```

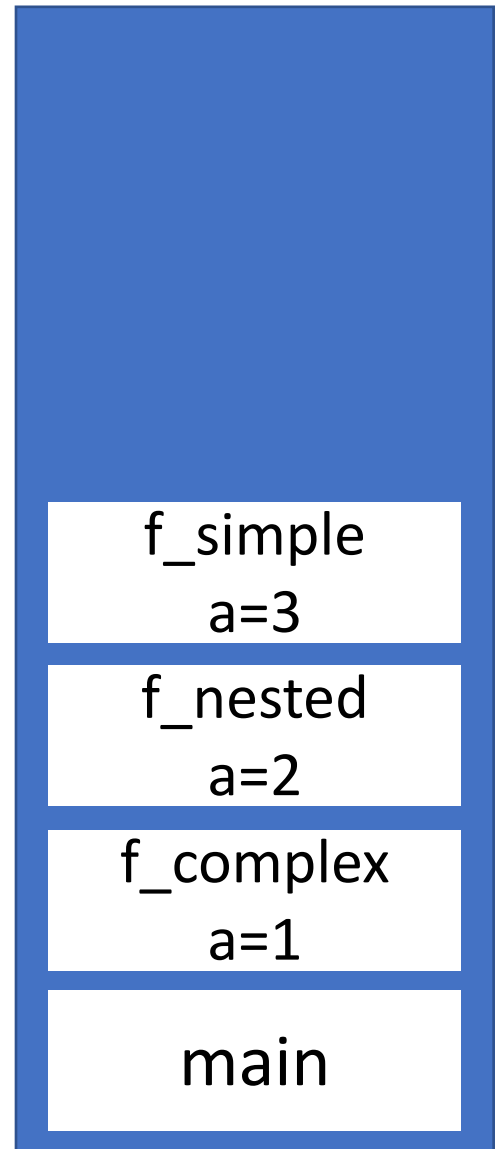
A terminale avremo

```
dentro il main, prima di chiamare f_simple  
funzione f_simple, a=1, ... ora ritorno
```

```
dentro il main, ho eseguito f_simple, sto chiamando f_complex  
funzione f_complex, a=1  
funzione f_complex, sto per chiamare f_nested  
funzione f_nested, a=2  
funzione f_nested, sto per chiamare f_simple  
funzione f_simple, a=3, ... ora ritorno
```

Quindi f_simple si chiude e verrà rimosso dallo stack

STACK



Esempio

Eseguiamo le istruzioni una ad una (si veda la freccia a sinistra)

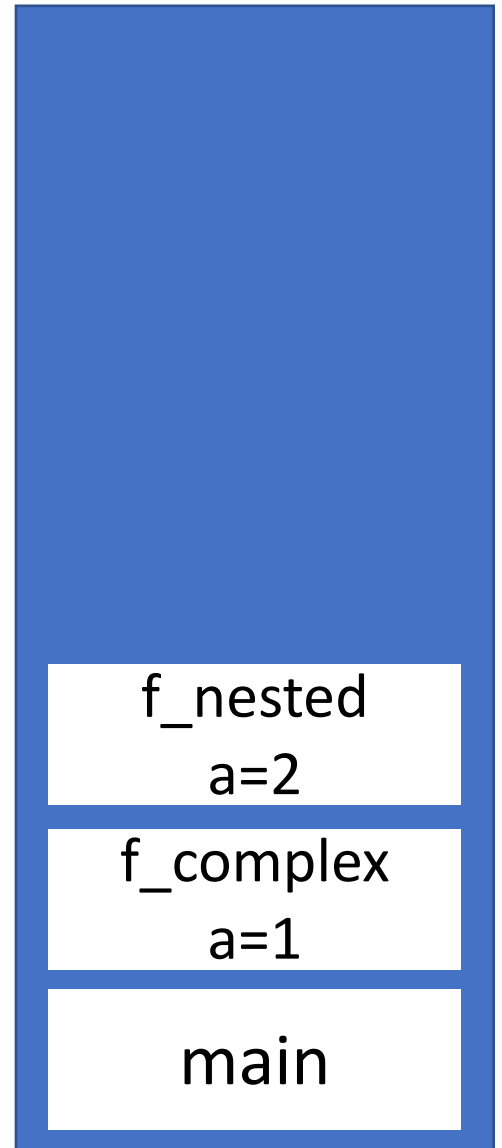
```
void f_nested(int a){  
    printf("\t\tfunzione f_nested, a=%d \n", a);  
    printf("\t\tfunzione f_nested, sto per chiamare f_simple\n\t\t");  
    f_simple(a+1);  
→ printf("\t\tfunzione f_nested, ho eseguito f_simple, ora ritorno\n");  
}
```

A terminale avremo

```
dentro il main, prima di chiamare f_simple  
funzione f_simple, a=1, ... ora ritorno  
  
dentro il main, ho eseguito f_simple, sto chiamando f_complex  
funzione f_complex, a=1  
funzione f_complex, sto per chiamare f_nested  
funzione f_nested, a=2  
funzione f_nested, sto per chiamare f_simple  
funzione f_simple, a=3, ... ora ritorno  
funzione f_nested, ho eseguito f_simple, ora ritorno
```

Quindi f_nested si chiude ed il corrispondente record verrà rimosso dallo stack

STACK



Esempio

Eseguiamo le istruzioni una ad una (si veda la freccia a sinistra)

```
void f_complex(int a){  
    printf("\tfunzione f_complex, a=%d \n", a);  
    printf("\tfunzione f_complex, sto per chiamare f_nested\n");  
    f_nested(a+1);  
→ printf("\tfunzione f_complex, ho eseguito f_nested, ora ritorno\n");  
}
```

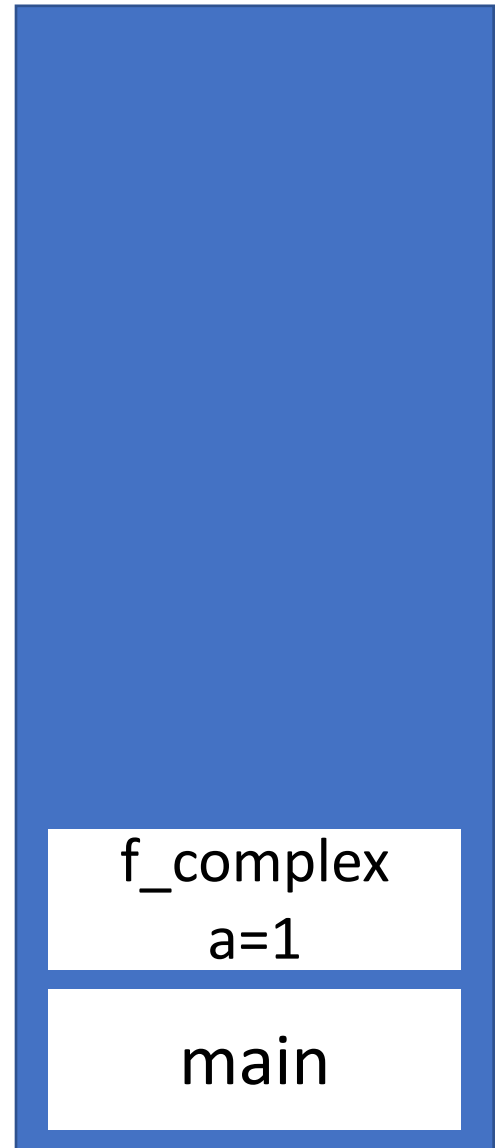
A terminale avremo

dentro il main, prima di chiamare f_simple
funzione f_simple, a=1, ... ora ritorno

dentro il main, ho eseguito f_simple, sto chiamando f_complex
funzione f_complex, a=1
funzione f_complex, sto per chiamare f_nested
funzione f_nested, a=2
funzione f_nested, sto per chiamare f_simple
funzione f_simple, a=3, ... ora ritorno
funzione f_nested, ho eseguito f_simple, ora ritorno
funzione f_complex, ho eseguito f_nested, ora ritorno

f_complex termina ed il controllo ritornerà al main

STACK



Esempio

Eseguiamo le istruzioni una ad una (si veda la freccia a sinistra)

```
int main(){  
    printf("\ndentro il main, prima di chiamare f_simple \n");  
    f_simple(1);  
    printf("\ndentro il main, ho eseguito f_simple, sto chiamando f_complex\  
f_complex(1);  
→ printf("\ndentro il main, ho eseguito f_complex, termina\n\n");  
}
```

A terminale vediamo

```
dentro il main, prima di chiamare f_simple  
funzione f_simple, a=1, ... ora ritorno
```

```
dentro il main, ho eseguito f_simple, sto chiamando f_complex  
funzione f_complex, a=1  
funzione f_complex, sto per chiamare f_nested  
funzione f_nested, a=2  
funzione f_nested, sto per chiamare f_simple  
funzione f_simple, a=3, ... ora ritorno  
funzione f_nested, ho eseguito f_simple, ora ritorno  
funzione f_complex, ho eseguito f_nested, ora ritorno
```

```
dentro il main, ho eseguito f_complex, termina
```

Al termine del main lo stack sarà vuoto

STACK

main

In modo analogo, per i blocchi avremo:

```
#include <stdio.h>
#include "variables.h"

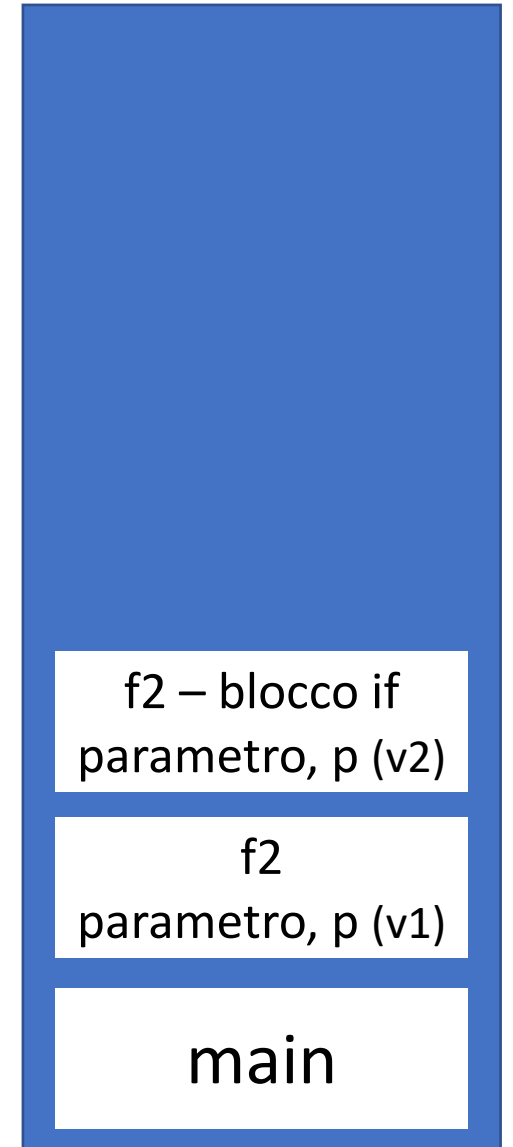
void f2(int parametro){
    p -=20; // Variabile globale
    printf("dentro f2 il valore di p=%f\n", p);

    float p = 5; //Non posso più accedere a p globale!!
    printf("dentro f2 il valore di p variabile locale è =%f\n", p);

    if (p>0){
→      float p = 20;
        printf("valore di p nel blocco if è =%f\n", p);
    }

    printf("dentro f2 dopo if,  variabile locale p è =%f\n", p);
}
```

All'uscita dell'if, il record in cima alla pila è eliminato...



Esempio

Step 1: Operating system invokes `main` to execute application

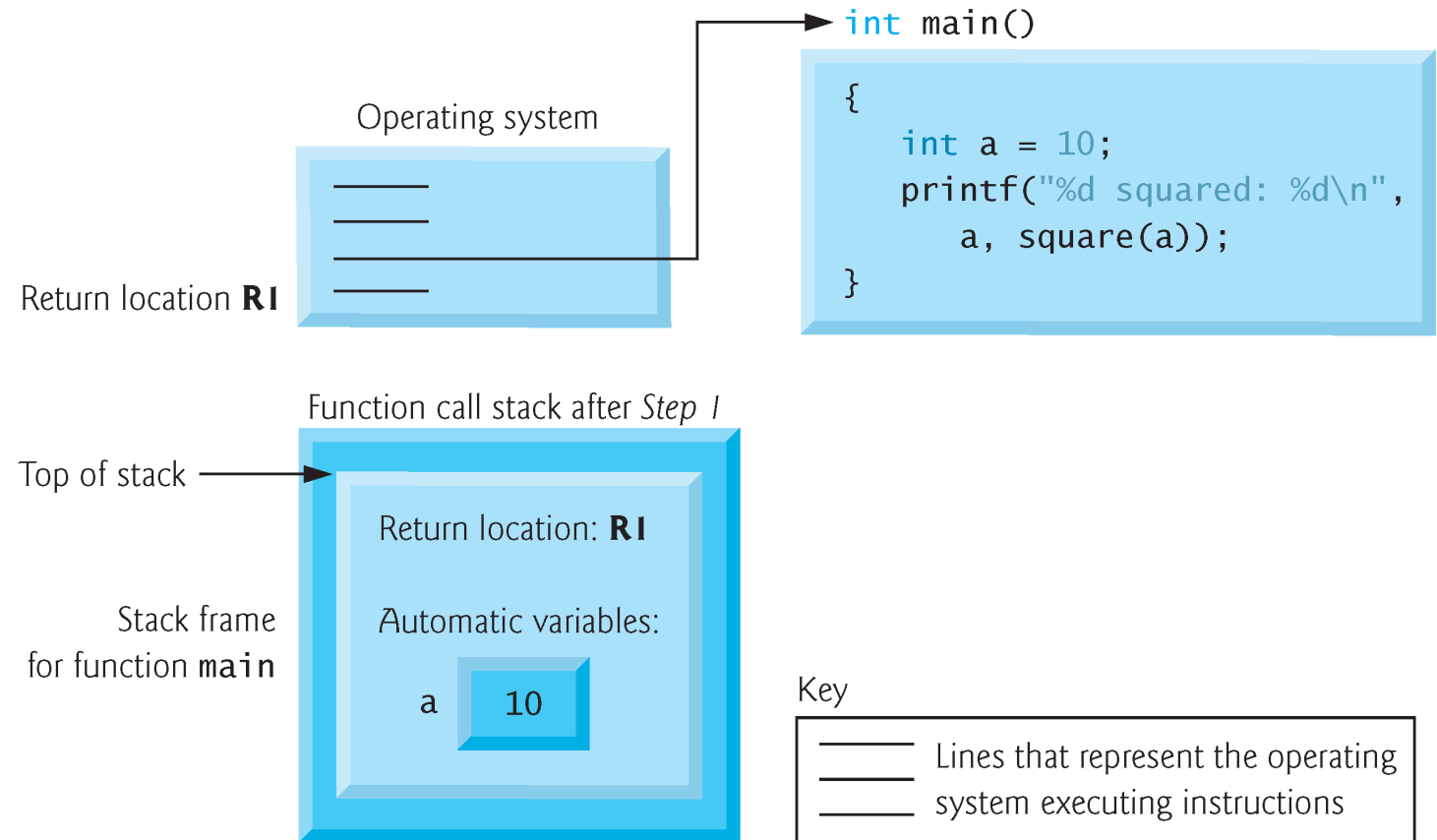


Fig. 5.7 | Function call stack after the operating system invokes `main` to execute the program.

Esempio

Step 2: `main` invokes function `square` to perform calculation

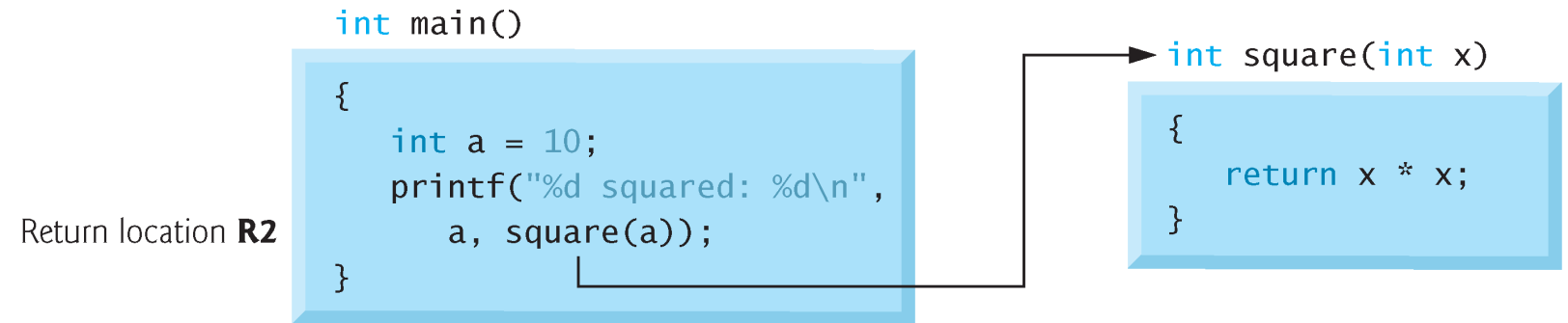


Fig. 5.8 | Function call stack after `main` invokes `square` to perform the calculation. (Part 1 of 2.)

Esempio

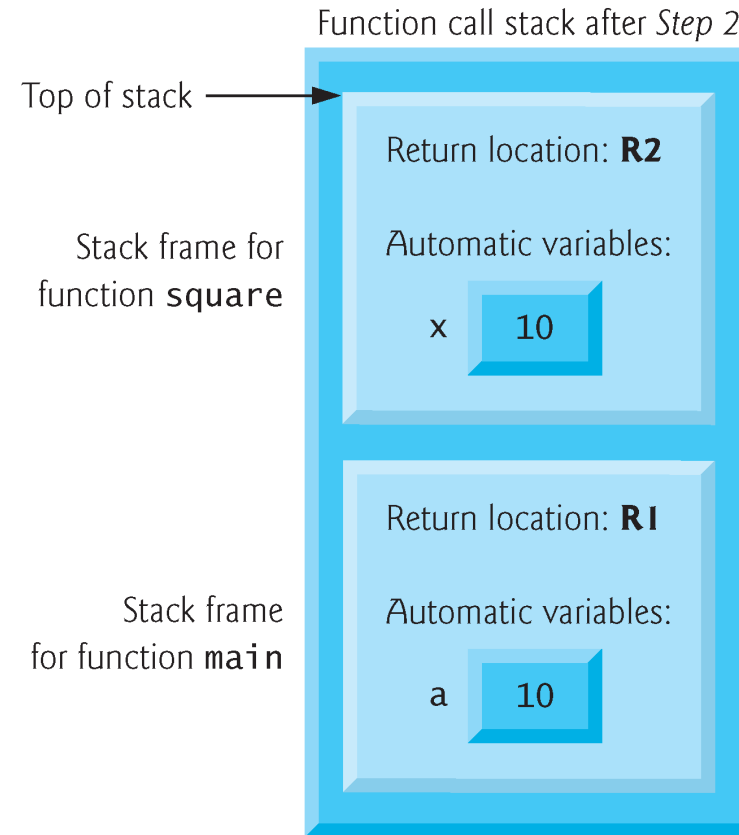


Fig. 5.8 | Function call stack after `main` invokes `square` to perform the calculation. (Part 2 of 2.)

Esempio

Step 3: `square` returns its result to `main`

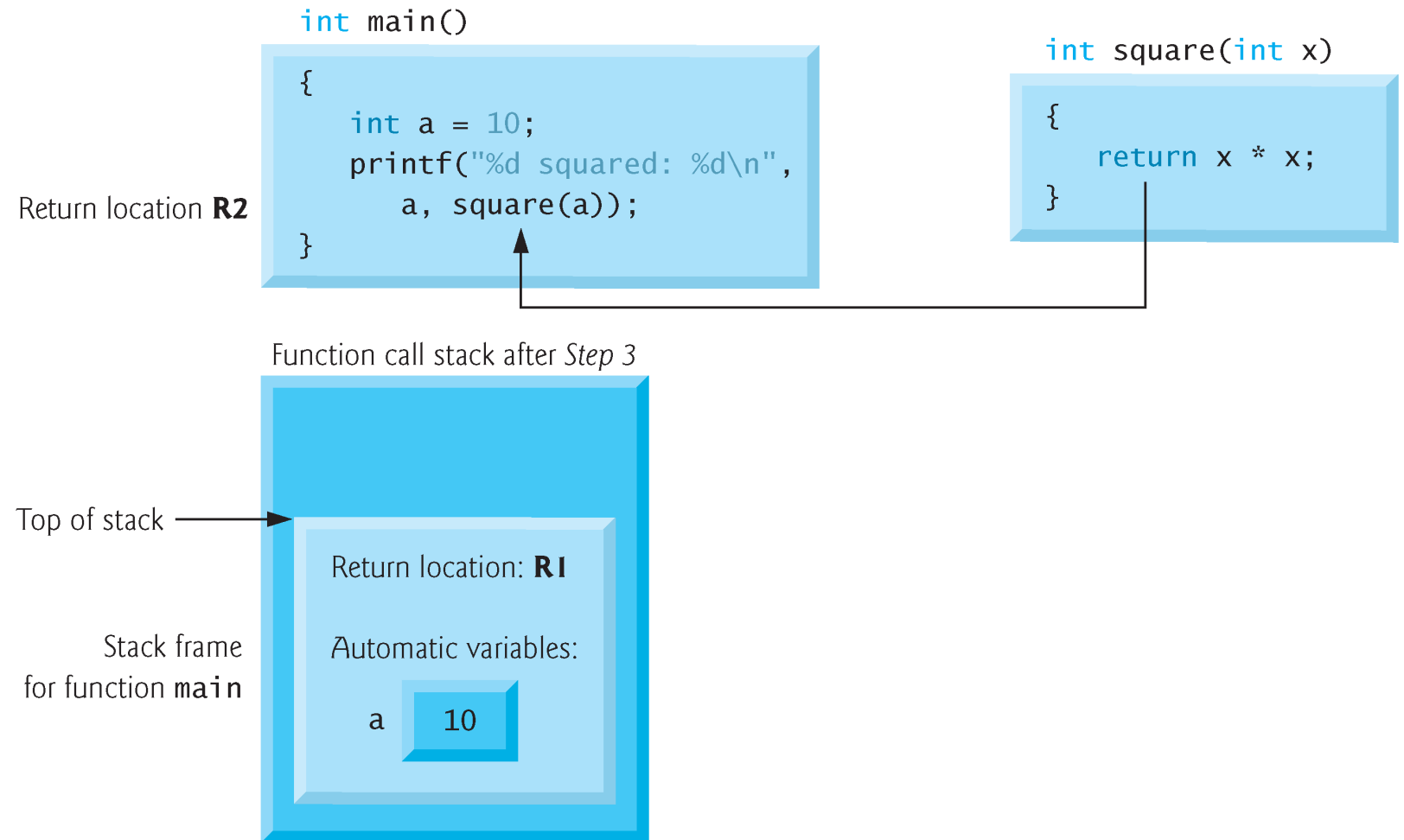


Fig. 5.9 | Function call stack after function `square` returns to `main`.

Memorie usate durante l'esecuzione di un programma

- Ad un programma in esecuzione è assegnato uno spazio di memoria.
- Tale memoria è suddivisa in aree atte a memorizzare specifiche informazioni: area per le **istruzioni**, area per i **dati**, area per lo **stack**, area per la memoria dinamica (quest'ultima è detta **heap**, in italiano memoria di mucchio).
- Le variabili globali sono nell'area di memoria destinata ai dati.
- L'**heap** contiene le strutture di memoria allocate dinamicamente (come vedremo a breve).

